

SVM & Logistic Regression for Wine-Quality Prediction

Project for the course Statistical Methods for Machine Learning at Università degli Studi di Milano

Author: Daniele Monti

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Environment

The project uses a tool called uv to manage Python packages. Instructions on how to install this software can be found at <https://docs.astral.sh/uv/getting-started/installation/>.

Once uv is installed, the environment can be reproduced by following these simple steps:

- navigate to the project's folder
- run the command "uv sync"
- activate the python virtual environment situated in the folder .venv

To execute a python script, use the command "uv run script_name.py".

All charts and plots for exploratory data analysis and model evaluation can be recovered by running the corresponding jupyter notebook.

Exploratory Data Analysis

Exploratory data analysis was conducted within the jupyter notebook 'data_analysis.ipynb'. There are 11 features, and the dataset contains a total of 6497 records, none of which contains a null value; missing data treatment is therefore unnecessary.

The quality has been transformed into a binary class as per instructions: quality greater or equal to 6 corresponds to the positive class, identified by the value 1, quality less than 6 corresponds to the negative class, identified by the value -1. The two classes thus formed are somewhat imbalanced, with the positive class comprising the 63.31% of the total and the negative comprising of the 36.69%. This issue will be addressed in the section on preprocessing.

There are 993 records which are present more than once, and a handful which appear as high as 8 or 7 times. This is not a result of the transformation of the quality into a binary class, as the same counts apply also for the non-transformed dataset. According to the research paper [1] by the original creators of the dataset:

“The data were recorded by a computerized system (iLab), which automatically manages the process of wine sample testing from producer requests to laboratory and sensory analysis. Each entry denotes a given test (analytical or sensory) [...]”

They also say:

“During the preprocessing stage, the database was transformed in order to include a distinct wine sample (with all tests) per row.”,

which leads me to the conclusion that rows with the exact same feature values corresponds to different taste tests rather than to different wines samples that happen to have the same feature values. If this is the case, it is surprising that in all of such cases, each taste test resulted in the same quality evaluation from the experts, but since the original authors decided to use the dataset without dropping the duplicated rows, I will do the same.

There is also an argument to be made that keeping the duplicated records better represents the underlying unknown distribution of the data, so dropping the records would be detrimental. At the same time, it is quite possible that some records seen during training will also be seen during testing, which is not desirable. This could be avoided by assigning a weight to each record proportional to the number of times it appears in the dataset and then eliminating the duplicates, but I did not want to overcomplicate the project.

Univariate distribution analysis shows that all of the features are approximately normally distributed, with several having a light skew (e.g. ‘volatile acidity’, ‘residual sugar’, ‘alcohol’). Some features, namely ‘free sulfur dioxide’, ‘volatile acidity’, ‘citric acid’, ‘residual sugar’, ‘chlorides’, ‘total sulfur dioxide’ and ‘density’ present several extreme outliers, for a total of 14 records affected. All the other features present a few (as is the case for ‘alcohol’ or ‘total sulfur dioxide’) or many milder outliers (as in the case of all the remaining features). Extreme outliers have been treated the same as every other during the preprocessing stage, though maybe it would be beneficial to outright remove them from the dataset.

Bivariate analysis shows that there are no extremely strong linear correlations between features, and there are only two cases where there is a relatively strong linear correlation, which are, unsurprisingly:

- positive, between ‘total sulfur dioxide’ and ‘free sulfur dioxide’
- negative, between ‘alcohol’ and ‘density’

In all other cases, the correlation coefficient is smaller than 0.54 in absolute value.

Preprocessing

The preprocessing utilities are inside the file ‘preprocessing.py’. They serve as to solve three main problems:

- outlier treatment
- scaling
- imbalance treatment

For every feature, a value x is considered an outlier if:

$$x < \mu - 3\sigma \vee x > \mu + 3\sigma$$

where μ is that feature’s mean and σ is that feature’s standard deviation. Any outlier value is then clipped to the lower or upper value. I chose this definition over the IQR based definition because many features have a rather long skew tail which is formed by a non indifferent number of records that would be cut short, accumulating too many records at the cutting point.

Each feature is scaled using scikit-learn StandardScaler, which applies the following transformation:

$$z = \frac{x - \mu}{\sigma}$$

where x is the value of the feature before scaling, μ is that feature's mean and σ is that feature's standard deviation. This shifts the data into a normal distribution of unit variance and zero mean. Scaling feature is particularly important for learning a linear classifier, because otherwise a feature with a bigger scale would be given more importance by the model weights.

Finally, as already mentioned, the dataset is moderately unbalanced towards the positive class. In order to help the model learn to better classify the negative class, an imbalance treatment technique known as SMOTETomek is applied using the imbalanced-learn Python package implementation. SMOTETomek consist in first applying the over-sampling technique SMOTE, which generates synthetic data point focusing on the region near the class boundaries, and then cleaning the result using the under-sampling technique based on Tomek links, which focuses on selecting the closest pairs of data points belonging to different classes and eliminating the data point belonging to the majority class.

Note that this preprocessing step is applied only on the training set.

Of course, care has been taken in first generating the values (e.g. μ and σ) using only the training set and then applying the same transformation separately to both the training and the test set. This is especially important when doing nested cross validation: the data is passed unprocessed into the inner loop, where for each fold data gets preprocessed fitting the clipper, the scaler and the rebalancer only using the current fold's training set. Once the inner loop is finished and we got the best value of the hyperparameters, the outer loop current fold's sets are still unprocessed, so we need to once again use the training set to fit the clipper, the scaler and the rebalancer. This is repeated for each loop of the outer cross validation procedure.

Models

Both the SVM model and the Logistic Regression model have been implemented using Pegasos, the Stochastic Gradient Descent (SDG) algorithm presented in [2]. I chose to implement the basic version and not the mini-batch version for ease of implementation and because it's the version presented during the course lecture. During the random drawing procedure I keep track of epochs, i.e. the number of times we extracted a number of samples equal to dimension of the training set. This is done in order to implement a stopping criterium, which is to stop the algorithm once a prefixed number of epochs *max_iter_no_changes* has passed without seeing an improvement in the value of the objective function within a prefixed tolerance *tol*. This was inspired by the scikit-learn implementation of the SGD procedure.

SVM

We wish to learn a linear predictor $h: X \rightarrow \{-1, 1\}$ such that $h(x) = \text{sgn}(w^T x + b)$, which given an input data point x will predict its label y . The idea at the core of SVMs is to pick the parameters w and b such that the geometric margin between the hyperplane $w^T x + b$ and the closest data points is maximized, while also minimizing the loss that we incur when a data point is misclassified. This objective can be expressed by the following objective function, which we want to minimize:

$$F(w, b) = \frac{\lambda}{2} \|w\|^2 + \frac{1}{m} \sum_{i=1}^m [1 - y_i (w^T x_i + b)]_+ \quad (1)$$

where λ is an hyperparameter controlling the strength of the regularization, m is the size of the training set and $[\cdot]_+$ is a function that returns its argument if it is positive and zero otherwise. The algorithm is a SGD algorithm, so after initializing $w = 0$, $b = 0$, for each update step t it performs the following operation:

- draw a random sample i
- approximate the objective function (1) with $F(w, b; i) = \frac{\lambda}{2} \|w\|^2 + [1 - y_i(w^T x_i + b)]_+$
- compute the sub-gradient with respect to w $\nabla_w = \lambda w - \mathbb{I}[y_i(w^T x_i + b) < 1] y_i x_i$, where $\mathbb{I}[y_i(w^T x_i + b) < 1]$ is the indicator function, and the sub-gradient with respect to b $\nabla_b = -\mathbb{I}[y_i(w^T x_i + b) < 1] y_i$
- update $w \leftarrow w - \frac{1}{\lambda t} \nabla_w$ and $b \leftarrow b - \frac{1}{\lambda t} \nabla_b$ (where the step size is carefully chosen in order to have a guarantee on the convergence rate for the algorithm)

until the stopping criterium is satisfied.

Computing the loss is the bottleneck for the algorithm, since it takes time $O(dm)$, where d is the number of features and m is the number of training data points, while all other operations take time $O(d)$. I sped up the computation using the python library numba in order to parallelize the cycle over all m training data points.

Logistic Regression

In this case we wish to learn the probability that an input data point belongs to the positive class instead of a directly predicting the class. In order to do so, we train a predictor $h: X \rightarrow \mathbb{R}$ and then we use $\sigma(g(x))$, where σ is the logistic function $\sigma(z) = \frac{1}{1 + e^{-z}}$, to predict the probability.

Since we are predicting a probability, instead of the hinge loss we use the logarithmic loss, defined as

$$l(y, \hat{y}) = \mathbb{I}[y = +1] \log \frac{1}{\hat{y}} + \mathbb{I}[y = -1] \log \frac{1}{1 - \hat{y}}$$

which can be reformulated as

$$l(y, g) = \log(1 + e^{-yg})$$

using the facts that $\hat{y} = \sigma(g(x))$, and $1 - \sigma(z) = \sigma(-z)$.

If we wish to then turn the prediction of the probability into an actual class prediction, all we have to do is set a threshold (in the project the threshold has been set to 0.5) on the probability value such that we predict positive if the probability is equal or higher and negative if the probability is smaller. This forms a decision boundary in the feature space, much like in the SVM case, given by the equation $\sigma(g(x)) = \text{threshold}$. If threshold is set to 0.5, this is equivalent to $g(x) = 0$, because the logistic function assumes value 0.5 if and only if its argument is equal to zero.

In our case, we want to learn a linear predictor, so we can express g as $g(x) = w^T x + b$. After adding a regularization term to prevent overfitting, the objective function to minimize is then

$$F(w, b) = \frac{\lambda}{2} \|w\|^2 + \frac{1}{m} \sum_{i=1}^m \log(1 + e^{-y(w^T x + b)}) \quad (2)$$

where once again λ is an hyperparameter controlling the strength of the regularization and m is the size of the training set.

We can use the exact same SGD algorithm presented in the SVM section, the only difference being the computation of the gradients, which now yields:

$$\begin{aligned}\nabla_w &= \lambda w - \sigma(-y_i(w^T x_i + b)) y_i x_i \\ \nabla_b &= -\sigma(-y_i(w^T x_i + b)) y_i\end{aligned}$$

Kernels

We now wish to expand the class of the learnable predictors beyond just linear predictors. In order to do so, we can create new features by taking non-linear combinations of the base features (more formally, we apply a feature map $\phi: \mathbb{R}^d \rightarrow H$, where H is a higher dimensional space). Learning a linear predictor in H is now akin to learning a non-linear predictor in the original space $\mathbb{R}^d$.

Such mappings ϕ quickly become computationally infeasible, but this problem can be avoided by using the so called “kernel trick”. If we consider a model in which the data points interact with each other only via inner product, then given a kernel function $K: X \rightarrow X$ which computes the inner product in H (that is $K(x, y) = \langle \phi(x), \phi(y) \rangle$), we can entirely bypass the computation of ϕ altogether by computing K instead.

Both (1) and (2) satisfy the hypothesis of the Representer theorem, provided that we substitute the inner products in the feature space $w^T x$ with the more general inner product of a RKHS $\langle \phi(w), \phi(x) \rangle$. Since the theorem holds, we can express the minimizers w^* of (1) and (2) as a linear combination of the projected training data points:

$$w^* = \sum_{i=1}^m \alpha_i y_i \phi(x_i) \quad \text{for some } \alpha \in \mathbb{R}^d$$

We can then limit our search space only to elements of the RKHS which can be represented by the coefficients α . Note that this allows us to learn a predictor even when the RKHS has infinite dimensions.

The extension of the models to kernelized form is achieved following the proposal found in [3], with the additional incorporation of the augmented kernel trick in order to easily learn the bias term. The standard Representer Theorem dictates that an explicit, unpenalized bias term b introduces a strict equality constraint $\sum_{i=1}^m \alpha_i y_i = 0$ into the dual space. This hard constraint is computationally incompatible with single-sample SGD updates. To eliminate this constraint, the bias is embedded directly into the RKHS by appending a constant feature dimension with value 1 to the implicit feature map:

$$\phi_{aug}(x) = \begin{bmatrix} \phi(x) \\ 1 \end{bmatrix}, \quad w_{aug} = \begin{bmatrix} w \\ b \end{bmatrix},$$

Applying the kernel trick to this augmented space yields:

$$K_{aug}(x, x') = \langle \phi_{aug}(x), \phi_{aug}(x') \rangle = \langle \phi(x), \phi(x') \rangle + (1 \cdot 1) = K(x, x') + 1$$

while the computation for the inner product between the predictor and a general data point x becomes:

$$\langle w_{aug}, \phi_{aug}(x) \rangle = \langle w, \phi(x) \rangle + (b \cdot 1) = \langle w, \phi(x) \rangle + b$$

This has the side effect of subjecting the bias to the regularization penalty, that is the SGD now minimizes $\frac{\lambda}{2} (\|w\|^2 + b^2)$. Geometrically, this restricts the decision boundary from drifting infinitely

far from the origin (which is not a problem since the data gets scaled before training), rendering the optimization problem entirely unconstrained and highly compatible with stochastic sampling.

The predictor w will be represented as $w = sv$, where s is a scalar value s and v is an array containing the coefficients $\alpha_i y_i$ for each training vector i . This allows for the scaling of the coefficients due to the regularization in constant time rather than in $O(m)$, where m is the number of training data points. An additional variable *norm_sq* will be used to cache the value of the squared norm of w in order to avoid having to expensively compute $\alpha^T K \alpha$ each time we have to evaluate the objective function.

I will now describe the kernelized version of the SGD algorithm, which is used both for the SVM and the Logistic Regression, since the only difference is the value of the (sub-)gradient.

- Save a copy of the input training data points for future use in the prediction function
- Compute the gram matrix which contains all the inner products of the chosen kernel for each pair of training data points
- Initialize the vector of coefficients *coeff* to 0, *norm_sq* to 0 and s to 1
- For $t = 1, \dots, T$ (T is an upper limit on the iterations obtained by multiplying m by a parameter with default value 1000):
 - extract a random training data point (x_t, y_t)
 - compute learning rate $\eta_t = 1 / \lambda t$
 - compute the inner product $\langle w, \phi(x_t) \rangle = s \langle v, \phi(x_t) \rangle$
 - compute $s \leftarrow (1 - 1/t)s$
 - modify the entry of v corresponding to the selected training data point $v \leftarrow v + (y_t \eta_t) / s$
 - update *norm_sq* using the fact that the norm after the update step will be equal to

$$\begin{aligned} \|w - \eta_t \nabla F(w)\|^2 &= \|w - \frac{1}{\lambda t}(\lambda w - C y_t x_t)\|^2 = \|(1 - \frac{1}{t})w + \eta_t C y_t x_t\|^2 = \\ &= (1 - \frac{1}{t})^2 \|w\|^2 + 2(1 - \frac{1}{t}) \eta_t C y_t \langle w, \phi(x_t) \rangle + (\eta_t C y_t)^2 \|x_t\|^2 \end{aligned}$$

where C is the coefficient of $y_t x_t$ in the (sub-)gradient of the loss

- make the final value of the coefficients be directly representing w by computing $coeff \leftarrow s * coeff$

All of these steps take $O(1)$ time, with the exception of the computation of the inner product, which takes time $O(m)$, and the computation of the loss each epoch to decide if we can do an early stop, which takes time $O(m^2)$, and is therefore once again the bottleneck. I implemented the same speedup of the computation using numba as in the linear case.

After the model has been trained, prediction can be obtained by computing the gram matrix between the training data points and the test data points, and then for each test data point x computing:

$$\langle w, \phi(x) \rangle = \langle \sum_i^m \alpha_i y_i \phi(x_i), \phi(x) \rangle = \sum_i^m \alpha_i y_i \langle \phi(x_i), \phi(x) \rangle = \sum_i^m \alpha_i y_i K(x_i, x)$$

Which takes $O(m)$ time. This could be improved by only storing the support vectors, that is only those vectors for which α_i is non zero, but I did not want to further complicate the code, since this is not a major problem because the dataset used only contains ~6000 records.

Hyperparameters and their tuning

All of the models possess a rather small number of hyperparameters:

- Both linear models only have λ as hyperparameter. This controls the trade-off between a smooth decision surface and the minimization of misclassified training examples, with higher values favoring the former and lower values favoring the latter (i.e., it controls the strength of the regularization)
- The Gaussian kernel has λ , which has the same role as in the linear models, and in addition it has γ , which defines how much influence a single training example has. The larger γ is, the closer other examples must be to be affected. In other words, the higher γ is, the closer the model will resemble a 1-NN classifier, with high risk of overfitting
- The Polynomial kernel has λ as all the other models, and in addition it has the degree d , which controls the maximum degree with which the original features will be combined in the RKHS. A higher degree means a more elaborate decision boundary, but it also means a higher risk of overfitting

Hyperparameter tuning is performed with the support of a python library called “optuna”, which implements a Bayesian optimization algorithm to search the space of hyperparameters. This is an efficient replacement to performing a grid search. The tuning is conducted inside a nested cross-validation framework, and the outer cross-validation procedure does the following for each fold:

- it splits the data into train and test set
- it finds the best parameters for the current train set by running the inner cross-validation procedure using only the current train set (which will be further split into train and dev set) for each try of possible hyperparameters values. The metric to be optimized is the cross-validated accuracy
- it trains the model again using the best value of the hyperparameters found and the entire train set

The metric reported at the end of the outer cross-validation loop is the accuracy cross-validated by the outer loop.

Evaluation

The evaluation of the models has been conducted using nested cross validation, setting the number of folds to 5 both for the outer and the inner cycle. Plots showing the evolution of loss, training and testing accuracy during the training on the training set of each outer fold (performed using the best hyperparameters found thanks to the inner loop) can be found by running the jupyter notebook “model_performance.ipynb”. The plots for the loss skip the first several hundred iterations in order to have a more reasonable scale.

The max number of overall iterations is capped to 200 epochs to limit training time, which was otherwise too long since it was repeated many times over during cross validation.

In all cases it is not surprising that the loss and both of the accuracy metrics present a highly unstable profile: this is caused by the fact that we are using a SGD procedure considering a single data point at a time instead of doing mini-batches.

DISCLAIMER: I found out very late that I did not insert a random shuffle of the training set both before applying the outer and the inner cross validation loop, so I did not have enough time to train again some of the models. The models affected are: SVM with Polynomial Kernel, Logistic Regression with Polynomial Kernel

The results and the analysis are therefore biased because of the static initial order of the dataset.

SVM with Polynomial Kernel

The model obtains an overall accuracy metric of 0.689078 on the test set and of 0.767377 on the train set. The model is not overfitting, because looking at the plots, train and test accuracy both grow at the same time, and there's no clear point in which training accuracy increases but test accuracy starts to decrease. It is somewhat surprising that one of the folds chose to use *degree* equal to 2 and it obtained almost the same accuracy as the fold that chose *degree* equal to 11. This could perhaps be explained by the low amount of available data, a problem that is exacerbated by the multiple split into train and test due to nested cross validation.

Logistic Regression with Polynomial Kernel

I lost the files containing the evolution of the metrics.

SVM with Gaussian Kernel

The model obtains an overall accuracy metric of 0.798368 on the test set and of 1.000000 on the train set. Even though the training accuracy is perfect while the testing accuracy is not, the model does not seem to be overfitting, since both appear to be following the same exact trend (although it's hard to tell because of the scale). What is very noticeable is that the early stopping criterium did not trigger for any of the folds, even for fold 4, where the value of the hyperparameters is quite different from the others. Looking at the data, this probably happens because the value of the objective function at the start is so high (varying between 10^7 and 10^{11}) that once the contribute of the sum of the losses is minimized, the decrease in the value of the objective function caused by the regularization is steep enough, and the tolerance of 0.001 low enough, that the early stopping criterium never kicks in.

Another thing of interest is that fold 4 has the best performance, so it may be that were the search over the best possible parameters given more time or had the space itself be allowed to be bigger, it would have turned out that values of λ and γ more similar to those of fold 4 would have come out on top as the best in more than one case.

Logistic Regression with Gaussian Kernel

The model obtains an overall accuracy metric of 0.800369 on the test set and of 0.999402 on the train set, which is once again ever so slightly better than its SVM counterpart; the difference is so small that it can easily be attributed to the random draw of how the folds were formed. There are nonetheless some major differences:

- the values of the hyperparameters are much closer to the ones found in fold 4 of the SVM model
- fold 1 and 5 present a clear decrease in test accuracy as training continues after a certain point, which indicates the presence of overfitting. For the other folds it is hard to tell, but that does not seem to be the case
- only fold 5 reached the end without stopping early

All of these factors combined seem to indicate that for the Gaussian kernel, logistic regression is preferable.

SVM with Linear Kernel

The model obtains an overall accuracy metric of 0.704018 on the test set and of 0.736780 on the train set. There is no overfit, but the performance is quite lower than either of the models that use the Gaussian kernel. There is nothing worth pointing out with regards to the plots, except that the first fold took a lot more time for some reason.

Logistic Regression with Linear Kernel

The model obtains an overall accuracy metric of 0.708790 on the test set and of 0.736598 on the train set. The model does not overfit, and the performance is on par with the SVM model.

Linear SVM

The model obtains an overall accuracy metric of 0.723716 on the test set and of 0.740267 on the train set. Folds 3, 4 and 5 show some mild overfit at the end of the training, but overall the performance is as expected on par with the Kernel version, the only difference being the speed of the fitting algorithm.

Linear Logistic Regression

The model obtains an overall accuracy metric of 0.728643 on the test set and of 0.745310 on the train set, putting it practically on par with all the other linear model. The only thing of notice is the fold number 5, in which the test accuracy is actually better than the training accuracy throughout the fitting.

Overall, both of the models seem to achieve very similar results when using the same kernel, although it could be argued that the hardcoded threshold of 0.5 for making a prediction out of the probability estimate of Logistic Regression should have been treated as a hyperparameter as well; this could have perhaps changed the Logistic Regression performance.

Bibliography

- [1] Paulo Cortez, António Cerdeira, Fernando Almeida, Telmo Matos, José Reis, "Modeling wine preferences by data mining from physicochemical properties" *Decision Support Systems*, Volume 47, Issue 4, (2009) Pages 547-553
- [2] Shalev-Shwartz, S., Singer, Y., Srebro, N. *et al.*, "Pegasos: primal estimated sub-gradient solver for SVM", *Math. Program.* **127**, 3–30 (2011)
- [3] A. Z. Zeyuan, C. Weizhu, W. Gang, Z. Chenguang and C. Zheng, "P-packSVM: Parallel Primal grAdient desCent Kernel SVM," *2009 Ninth IEEE International Conference on Data Mining*, Miami Beach, FL, USA, 2009, pp. 677-686